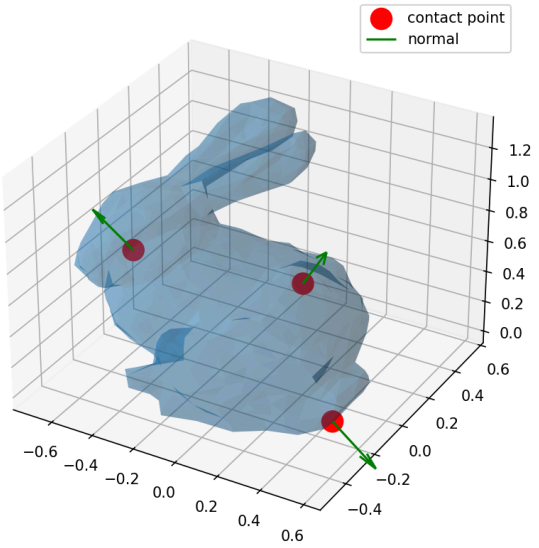
COMP 562 Project 2 Report

Cindy Nguyen

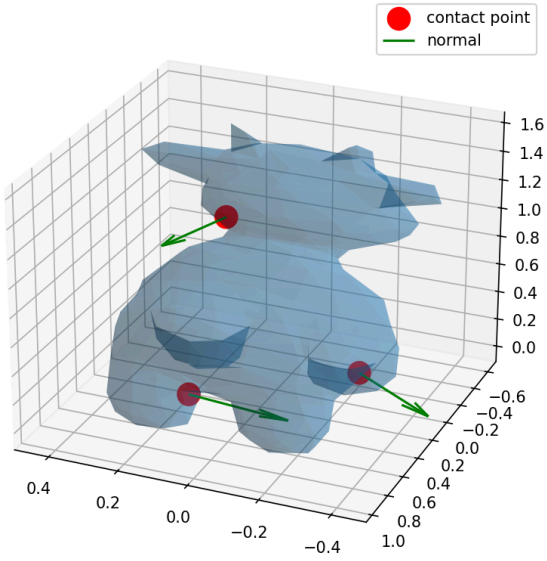
Task 2

Q. For each one of the mesh models (bunny, cow, and duck), given a three-contact grasp=[0, 249, 484], calculate the L1 quality when λ = 0.2, 0.5, and 1 respectively. In the report, 1) show a visualization of the grasp on each model; 2) provide a table summarizing the obtained grasp qualities; and 3) provide your insights on how the value of λ would affect the grasp quality calculation.

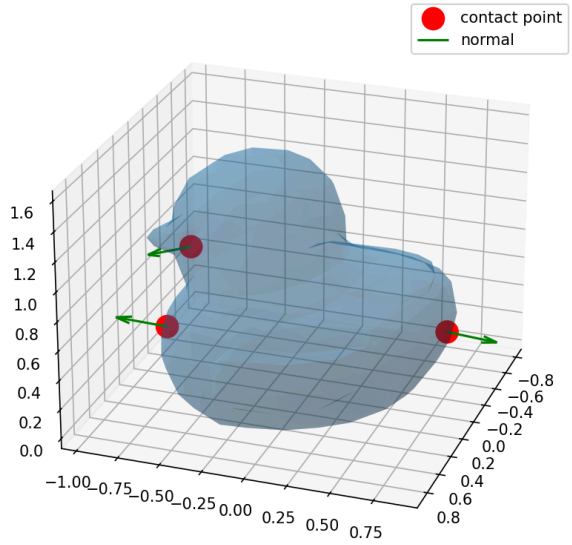
- Bunny:



- Cow:



- Duck:



Grasp Quality Table

lambda (λ)	Bunny	Cow	Duck
0.2	0.002751	-0.186112	-0.004202
0.5	0.006873	-0.422286	-0.009888
1.0	0.013704	-0.670294	-0.016677

lambda (λ) scales the torque dimensions of the 6D wrench space relative to the force dimensions. Since the wrench magnitude is defined as $\|w\| = \sqrt{(\|f\|^2 + \lambda\|\tau\|^2)}$, multiplying the torque columns by λ before building the convex hull effectively stretches or compresses the wrench space along the torque axes.

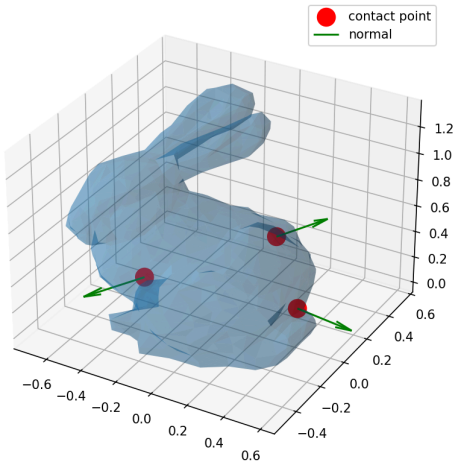
When λ is small, torques are downweighted so the convex hull is evaluated mainly based on the force components, making the quality score less sensitive to whether the grasp can resist rotational disturbances. When λ is large, torques are upweighted so the grasps that generate unbalanced torques are penalized more heavily, reducing the quality score.

As a result, a grasp that is marginally stable (Q slightly above 0) at λ = 1.0 may appear more stable at λ = 0.2 (torque resistance matters less) or less stable at higher λ. The choice of λ should reflect the physical application: a higher λ for tasks requiring precise rotational control, and a lower λ for tasks focused on translational movement.

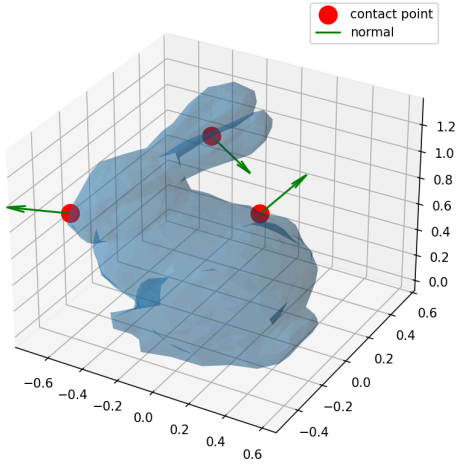
Task 3

Q. Set λ = 1.0 and thresh=0.01, please provide five different grasps (each grasp has three contacts) for the bunny model. In the report, please list the grasp contacts, visualize the grasps, and provide statistical analysis of the qualities of sampled grasps, e.g., histograms.

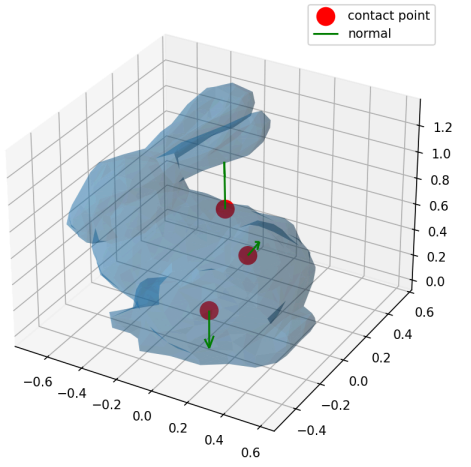
The grasp: [np.int32(290), np.int32(561), np.int32(793)]
The contact points of the stable grasp:
[[0.50017196 -0.1354416 0.38482182]
 [-0.19599094 -0.35188464 0.52469816]
 [0.10130739 0.28516904 0.41446876]]



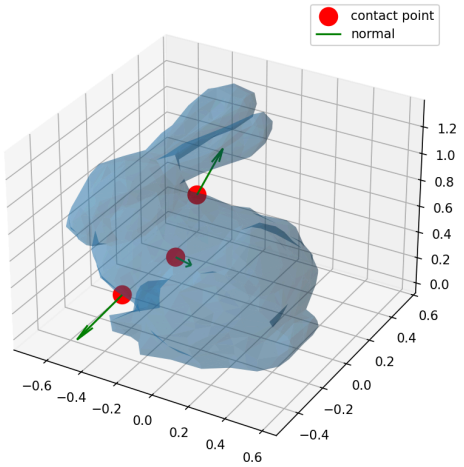
The grasp: [np.int32(883), np.int32(429), np.int32(225)]
The contact points of the stable grasp:
[[-0.32596791 0.34431057 0.96724345]
 [-0.66914852 -0.29424605 0.75694706]
 [0.06615652 0.18098362 0.64888972]]



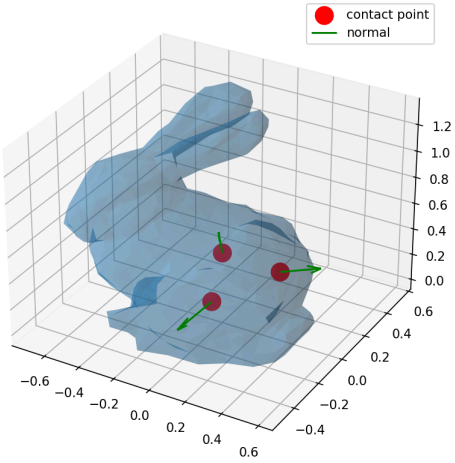
The grasp: [np.int32(28), np.int32(756), np.int32(979)]
The contact points of the stable grasp:
[[-0.06976518 -0.02509393 0.01511296]
 [0.29315521 -0.2334868 0.76785674]
 [-0.03175384 0.05157337 0.7532475]]



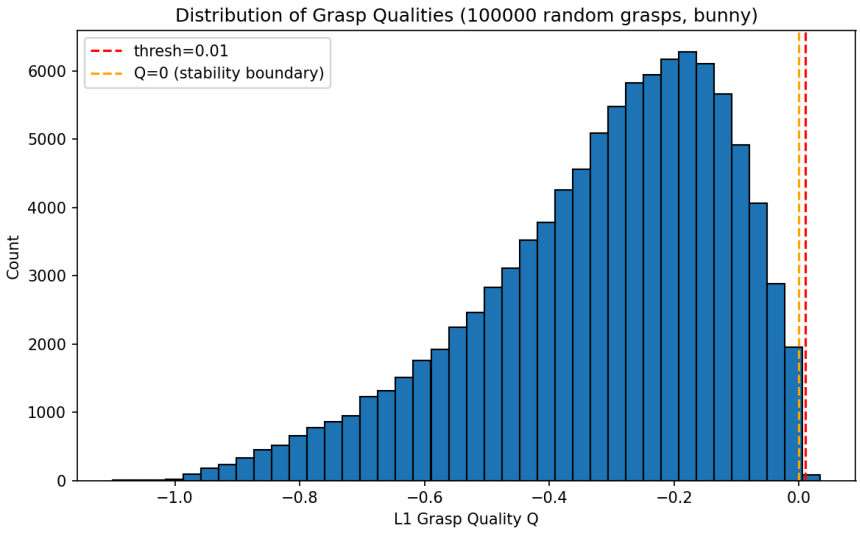
The grasp: [np.int32(488), np.int32(115), np.int32(414)]
The contact points of the stable grasp:
[[-0.36469023 -0.30684739 0.28866309]
 [-0.34950013 0.08525734 0.2490564]
 [-0.22483728 0.08674117 0.77514082]]



The grasp: [np.int32(289), np.int32(746), np.int32(1048)]
The contact points of the stable grasp:
[[0.44270916 -0.17841528 0.65194001]
 [0.23621984 -0.41716131 0.55408452]
 [-0.13058797 0.18644104 0.25688588]]



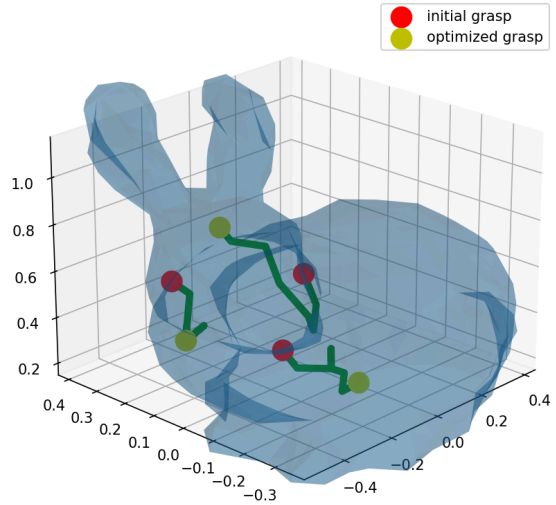
Grasp Quality Histogram



Fraction of grasps above thresh = 0.01: 40/100000 = **0.04%**
This statistical analysis shows that most of the generated grasps are actually unstable and only very few (0.04%) are above the threshold value.

Task 4

Q. On the bunny model, with λ = 1.0, please optimize the grasp=[80, 165, 444] and visualize the optimization trajectory in the report. Please discuss: how many grasp quality evaluations were needed in this optimization process? If you randomly sample grasps, what is the probability of obtaining a grasp of your optimized quality?



The quality of the given initial grasp: -0.128704
The quality of the optimized grasp: 0.022332

Trajectory length (steps accepted): 8
  grasp=[80, 165, 444], neighbor combo count ≈ 1680
  grasp=[np.int64(48), np.int64(651), np.int64(140)], neighbor combo count ≈ 3360
  grasp=[np.int64(224), np.int64(137), np.int64(504)], neighbor combo count ≈ 1560
  grasp=[np.int64(418), np.int64(552), np.int64(545)], neighbor combo count ≈ 1820
  grasp=[np.int64(498), np.int64(137), np.int64(504)], neighbor combo count ≈ 2080
  grasp=[np.int64(516), np.int64(552), np.int64(531)], neighbor combo count ≈ 1540
  grasp=[np.int64(521), np.int64(572), np.int64(499)], neighbor combo count ≈ 1680
Total evaluations ≈ **15020**

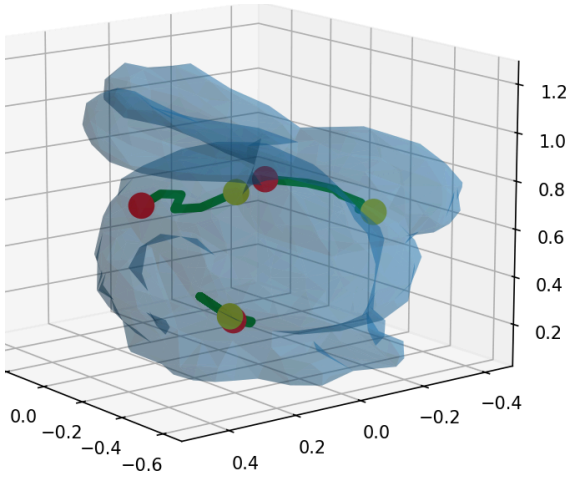
Number of evaluations: At each optimization step, local_optimal() evaluates every combination in the Cartesian product of the neighbor sets of all three contacts. With η=1 and a bunny face having around 15-25 vertex-adjacent neighbors, each step evaluates on the order of 15^3 to 25^3 ≈ 3,000-15,000 candidate grasps. Across 8 accepted steps in the optimized trajectory, the total number of eval_Q() calls is approximately 15020 evaluations.

Probability via random sampling: The optimized grasp quality was Q* = 0.022332 ≈ 0.022. To estimate the probability of reaching this quality by random sampling, we repeat the histogram experiment from Task 3 and set thresh = 0.022. Sampling 100000 random grasps, 8 achieved Q ≥ 0.022, giving the very tiny probability of 8/100000 = **0.01%**. This demonstrates the optimizer's significant efficiency advantage at finding the best grasp over blind random search.

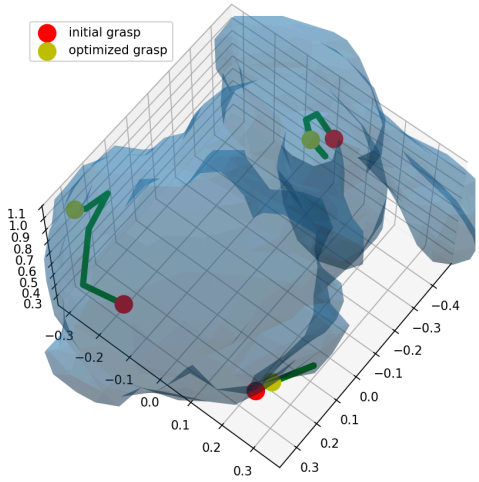
Task 5

Q. On the bunny model, with r = 0.5 and 1.0 respectively, please first sample a reachable grasp and optimize this grasp under the reachability constraint. Feel free to test with different initially sampled grasps. For each r, select one optimized trajectory to visualize in the report. Please provide a pseudo-code of your constraint-based optimization algorithm, and discuss potential improvements you can add to your algorithm.

r = 0.5:
The quality of the given initial grasp:
-0.421934
The quality of the optimized grasp: 0.011432



r = 1.0:
The quality of the given initial grasp:
-0.290691
The quality of the optimized grasp: 0.028329



Pseudocode

```
def optimize_reachable_grasp(mesh, r=0.5):
    traj ⇐ empty list

    def reachable(grasp):
        con_pts ⇐ get_centroid_of_triangles(mesh, grasp)
        centroid ⇐ mean of con_pts
        avg_dist ⇐ means of norm(con_pts - centroid)
        return avg_dist < r

    grasp ⇐ None
    while grasp is None do
        candidate ⇐ list of random.choice(len(mesh.faces), size=3,
replace=False))
        if reachable(candidate) then
            grasp ⇐ candidate
```

```
    traj ⇇ [list(grasp)]
    curr_grasp ⇇ list(grasp)
    curr_Q ⇇ eval_Q(mesh, curr_grasp)

    while True do
        nbr_sets ⇇ list of find_neighbors(mesh, face_id, eta=1) for
face_id in curr_grasp

        G_opt ⇇ None
        Q_max ⇇ curr_Q

        for combo in it.product(*nbr_sets) do
            candidate ⇇ list(combo)

            if (duplicate contacts in candidate) or not
reachable(candidate) then
                continue

            q ⇇ eval_Q(mesh, candidate)
            if q > Q_max then
                Q_max ⇇ q
                G_opt ⇇ candidate

        if G_opt is None then
            break

        curr_grasp ⇇ G_opt
        curr_Q ⇇ Q_max
        append list(curr_grasp) to traj

    return traj
```

Potential Improvements:

- Larger neighborhood order (η > 1): Using η=1 gives a narrow search radius per step. Increasing η to 2 or 3 allows larger moves per iteration, potentially escaping shallow local optima at the cost of more evaluations per step.
- Random restarts: The algorithm is a greedy local search and will converge to the nearest local optimum from the initial sample. Running the full optimization multiple times from different reachable starting grasps and returning the best result significantly reduces sensitivity to the starting point.
- Stochastic search: Instead of exhaustively evaluating the full Cartesian product (scales as $O(|\text{neighbors}|^m)$), randomly subsample the candidate combinations. This trades completeness for speed and allows larger η values to be used affordably.